Go ч.2

Указатель

- Указатель - это значение, хранящее адрес блока памяти. Размер указателя зависит от архитектуры: 8 байт на amd64, 4 байта на arm32.
- Значением по умолчанию для указателя является nil - это не NULL из C, а полноценное типизированное значение.
- Оператор & возвращает адрес переменной. Компилятор Go при этом выполняет escape analysis - если адрес переменной «убегает» из текущего scope (например, возвращается из функции), переменная размещается в heap, а не в stack.

```go
var x int
pntr := &x
fmt.Println(pntr) // 0xc00001c080
```

Указатель

- Оператор * выполняет разыменование (dereferencing) - получение значения по известному адресу. Это единственный способ «мутировать на расстоянии» в Go - в языке нет ссылок (references) и конструкций ref/out, как в C# или C++

```
var x int
pntr := &x
x = 64
fmt.Println(*pntr) // 64
```

Указатель

- Можно присвоить новое значение переменной, используя указатель

```
*pntr = 31
fmt.Println(*pntr) // 31
fmt.Println(x) // 31
```

Указатель

- В Go также выделяется отдельный тип данных uintptr, который представляет собой целочисленный тип данных для хранения числового значения адреса
- Его размер достаточен для хранения значения указателя
- Garbage Collector НЕ отслеживает uintptr, если объект доступен только через uintptr, GC может его собрать

```
var pntr1 *int // типизированный указатель, отслеживается GC
var pntr2 uintptr // числовое значение адреса, НЕ отслеживается GC
fmt.Println(pntr1) // <nil>
fmt.Println(pntr2) // 0
```

Указатель

- Как было сказано выше, значением по умолчанию для указателя является nil.
- При использовании типа данных uintptr значением по умолчанию является 0, так как это целочисленный тип данных и используется для хранения указателя.

nil safety

- Разыменование nil-указателя вызывает panic с сообщением «invalid memory address or nil pointer dereference».
- Нужно всегда проверять указатель перед разыменованием, особенно при работе с указателями из внешних API.

```
var p *int
// *p = 42 // PANIC - invalid memory address or nil pointer dereference

// Безопасный доступ:
if p != nil {
    fmt.Println(*p)
}
```

Функция new()

- Функция new(T) - синтаксический сахар: выделяет память для значения типа T, заполняет zero value и возвращает *T.
- Это полный эквивалент объявления переменной с взятием адреса. Однако new() не позволяет инициализировать значение - для структур предпочитайте & с литералом

```
var intValue int
p1 := &intValue    // способ 1 объявление + взятие адреса
p2 := new(int)     // способ 2 new() - эквивалентно
fmt.Println(p1)    // 0xc00001c080
fmt.Println(p2)    // 0xc00001c088
```

Массив

- Массив — это коллекция однотипных значений фиксированной длины:

```
var array [3]int
var defArr [3]string = [3]string{"one", "two", "three"}
```

- Длина массива указывается в квадратных скобках. Если не заполнить массив при инициализации, то массив будет состоять из нулевых значений данного типа. Длина массива является частью его типа:

```
nums := [5]int{} // [0, 0, 0, 0, 0]

var nums [5]int // [0, 0, 0, 0, 0]
```

Массив

- При попытке инициализации элементов за границей массива выходит ошибка компиляции:

```
nums := [1]int{1,2} // array index 1 out of bounds [0:1]
```

- Чтение и запись элементов массива происходит через квадратные скобки:

```
nums := [3]int{1, 2, 3}

fmt.Println(nums[1]) // 2

nums[2] = 33

fmt.Println(nums) // [1, 2, 33]
```

Массив

- Примеры инициализации:

```
var a = [3]int{1, 2, 3}
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3}
d := [3]int{1: 12}

fmt.Println(a) // [1 2 3]
fmt.Println(b) // [1 2 3]
fmt.Println(c) // [1 2 3]
fmt.Println(d) // [0 12 0]
```

- В случае, если потребуется объявить массив с заранее неизвестным количеством элементов, есть возможность определить это при объявлении при помощи оператора «...»

Массив

- Если мы создадим два массива в Go с разными длинами, то два массива будут иметь разные типы, так как длина массива в Go, входит в его тип:

```
a := [3]int{}
b := [2]int{}

// (a) [2]int и (b) [3]int - разные типы
```

Массив

- Иногда массивы могут оказаться слишком длинными для записи в одну строку, в этом случае Go позволяет записывать их в несколько строк:

```
x := [5]int{
    98,
    93,
    77,
    82,
    83,
}
```

Массив

- Нумерация элементов массива начинается с 0. При попытке чтения/записи элементов за границей массива выходит ошибка компиляции:

```
words := [2]string{}

words[2] // invalid array index 2 (out of bounds for 2-element array)
```

Массив

- Массивы в Go передаются по значению, следовательно, любое изменение внутри функции не влияет на исходный массив:

```go
package main

import (
    "fmt"
)

func main() {
    a := [3]int{1,2,3}

    modifyArr(a)

    fmt.Println(a) // 1, 2, 3
}

func modifyArr(nums [3]int) {
    nums[0] = 35
}
```

Массив

- В Go есть встроенная функция len(), которая возвращает длину массива:

```
fmt.Println(len([5]int{1,2,3})) // 5
fmt.Println(len([10]int{})) // 10
```

- Так как массивы инициализируются с фиксированной длинной, то функция len() всегда возвращает одно и то же значение.

Массив

- Итерация по массиву через for range возвращает (индекс, значение), range создаёт КОПИЮ каждого элемента!

```
grades := [5]int{98, 93, 77, 82, 83}
for i, v := range grades {
    fmt.Printf("grades[%d] = %d\n", i, v)
}
```

Слайс/Срез

- На практике не часто сталкиваешься с массивами из-за ограниченной длины при строгой типизации. Вместо этого повсеместно используются слайсы.
- Слайс - это массив неопределенной длины (или динамический массив).
- Глядя на объявление слайса, может показаться, что он является тем же массивом, хотя это не так.

```
var nums = []int{1,2,3}

nums := []int{1,2,3}
```

Слайс/Срез

- Чтобы было нагляднее, лучше заглянуть в исходный код и найти описание структуры слайса.
- Файл: src/runtime/slice.go

```
type slice struct {
  array unsafe.Pointer // указатель на backing array
  len int // текущая длина (кол-во элементов)
  cap int // вместимость (размер backing array)
}
```

Слайс/Срез

- Как видно по коду, слайс представляет собой структуру. В его основе лежит массив, указатель на который хранится в слайсе. Кроме этого, есть два атрибута: len - длина и cap - вместимость.
- Длина показывает, какое количество элементов в данный момент находится в слайсе. В отличие от массива, слайс может изменять свой размер в процессе выполнения программы. Размер слайса можно узнать при помощи уже известной встроенной функции len().

Слайс/Срез

- Вместимость среза показывает, сколько еще элементов можно добавить до того, как Go осуществит новую аллокацию памяти.
- Когда объявляется срез, Go производит выделение памяти, которой будет достаточно для хранения исходного массива указанной размерности.
- Если известна длина, то, как и в случае с массивом, Go заполняет массив нужным количеством элементов со значением по умолчанию для типа, который был указан в выражении инициализации слайса. При этом значение атрибута len равно количеству проинициализированных элементов, а cap — значению len. Узнать, какая вместимость в данный момент у слайса, можно с помощью функции cap().

Слайс/Срез

- len показывает текущее количество элементов.
- cap показывает, сколько элементов поместится до новой аллокации.
- Когда len == cap, следующий append вызовет реаллокацию - создание нового, большего массива и копирование данных.

```
var s []int
fmt.Println(len(s)) // 0
fmt.Println(cap(s)) // 0
```

Слайс/Срез

- Чтение и запись осуществляется точно так же как в массивах:

```
nums := []int{1,2,3}

nums[2] // 3

nums[0] = 10 // [10, 2, 3]

// с помощью оператора : можно получить элементы от нижней до верхней границы
nums[1:3] // [2, 3]
nums[:2] // [10, 2]
nums[2:] // [3]
```

Слайс/Срез

- Стоит отметить, что, так как в основе слайса лежит указатель на массив, на один и тот же базовый массив могут ссылаться разные слайсы, причем слайсы могут пересекаться и ссылаться на одни и те же элементы.

```
s := []string{"e", "l", "e", "m", "e", "n", "t"}
s1 := s[:4]
s2 := s[1:6]
fmt.Println(s1, s2) // [e l e m] [l e m e n]
s1[1] = "L"
fmt.Println(s, s2) // [e L e m e n t] [L e m e n]
```

- В слайсе s1 мы изменили элемент с индексом 1, и в результате все слайсы, которые ссылались на один и тот же базовый массив, изменились

Слайс/Срез

- В слайсы можно добавлять элементы с помощью встроенной функции func append(slice []Type, elems ...Type) []Type, которая возвращает новый слайс с добавленным элементом:

```
words := []string{"hello"}

words = append(words, "world") // ["hello", "world"]
```

Слайс/Срез

- Если понадобится вставить элементы одного слайса в другой, то можно применить оператор ... , который может упаковать или распаковать слайс и передать его элементы в качестве аргументов в функцию

```
s := []int{1, 2, 3}
p := []int{4, 5, 6}
s = append(s, p...)
fmt.Print(s) //[1 2 3 4 5 6]
```

Слайс/Срез и функция make()

- make() - создание слайса с предаллокацией.
- Позволяет задать начальную длину и вместимость, что критично для производительности, если знаете примерный размер, предаллокация избежит множественных grow операций.

```
// len = 5. Массив сразу будет заполнен 5-ю нулевыми значениями
nums := make([]int, 5, 5) // [0, 0, 0, 0, 0]

// len = 0, но cap = 5. Массив будет пустым, однако заполнение слайса через append будет
// эффективным, потому что в памяти уже выделен массив нужной длины
nums := make([]int, 0, 5) // []
```

Слайс/Срез

- Передача слайса как аргумента функции происходит хитро. Длина и вместимость передаются по значению, но массив значений передается по ссылке. Вследствие этого получается неявное поведение - добавленные элементы не сохранятся в исходный слайс, но изменение существующих останется:

Слайс/Срез

```go
package main

import (
    "fmt"
)

func main() {
    nums := []int{1, 2, 3, 4, 5}

    modifySlice(nums)

    fmt.Println(nums) // [1 2 10 4 5]
}

func modifySlice(nums []int) {
    nums[2] = 10 // элемент будет и в исходном слайсе
    nums = append(nums, 6) // элемент не добавится в исходный слайс
}
```

Слайсы и цикл for

```go
nums := make([]int, 0, 10)

// начиная с 0; пока i меньше 10; инкрементим i после каждого шага
for i := 0; i < 10; i++ {
    nums = append(nums, i)
}

fmt.Println(nums) // [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Слайс/Срез

Для обхода коллекции в Go есть "синтаксический сахар" range. Эта конструкция обходит слайс, возвращая индекс и элемент на каждом шаге:

```
names := []string{"John", "Harold", "Vince"}

// i — это индекс, name — это значение на текущем шаге цикла
for i, name := range names {
    fmt.Println("Hello ", name, " at index ", i)
}
```

Функции

- Функции в Go объявляются через ключевое слово func:

```
func multiply(x int, y int) int {
    return x * y
}
```

- Объявление функции состоит из следующих частей:
  - ключевое слово func
  - название функции multiply
  - входящие аргументы (x int, y int). Тип аргументов пишется после названия переменной. Если несколько аргументов подряд имеют одинаковый тип, то можно написать сокращенно (x, y int)
  - тип возвращаемого значения int
  - тело функции { return x * y }

Функции

Функции именуются в camelCase. Если первая буква заглавная, то функция экспортируемая (публичная) и доступна в других пакетах. Функции с маленькой буквы используются только в рамках текущего пакета:

```
package math

// публичная функция, можно вызвать извне как math.Multiply(5,7)
func Multiply(x int, y int) int {
    ...
}

// приватная функция, извне не вызвать
func divide(x int, y int) int {
    ...
}
```

Функции

Из одной функции можно возвращать несколько значений. Чаще всего это используется для возвращения ошибок:

```
package math

import "errors"

func divide(x, y int) (int, error) {
    if y == 0 {
        return 0, errors.New("cannot divide on zero")
    }

    return x / y, nil
}
```

Функции

Возвращаемые значения могут быть именованными:

```
func multiply(x, y int) (res int) {
    res = x * y
    return
}
```

Функции

Использовать именованные возвращаемые аргументы - плохая практика. На это есть несколько причин:

- увеличенное расстояние между объявлением и использованием переменной ведет к сложности в чтении и понимании кода
- переменная может быть несколько раз переопределена в теле функции, что приводит к неожиданному поведению
- пустой return неявно возвращает аргументы

Функции

Естественно, код состоит не только из функций, чаще всего используется множество внешних библиотек. Чтобы использовать внешнюю функцию, нужно указать пакет и через . вызвать публичную функцию. Например:

```
import "fmt"

func myPrint(msg string) {
    // пакет.функция
    fmt.Println(msg)
}
```

Копирование слайсов

Допустим, в вашей функции происходят изменения элементов, но вы не хотите затронуть входной слайс. В языке есть встроенная функция func copy(dst, src []Type) int, которая копирует слайс src в слайс dst и возвращает кол-во скопированных элементов:

```
nums := []int{1,2,3,4,5}

// важно инициализировать слайс той же длины
numsCp := make([]int, len(nums))

copy(numsCp, nums)

fmt.Println(numsCp) // [1,2,3,4,5]
```

Копирование слайсов

Почему мы не можем просто перезаписать слайс в другую переменную и изменять ее? Как и с функциями, при присваивании слайса к переменной, копируется только длина и вместимость, но массив передается по ссылке:

```
nums := []int{1,2,3,4,5}

numsCp := nums

// исходный слайс nums тоже будет изменен
numsCp[0] = 10

fmt.Println(nums) // [10,2,3,4,5]
```

Копирование слайсов

Существует распространенная ошибка, когда пытаются скопировать слайсы различной длины. В этом случае элементы, выходящие за рамки слайса dst, не будут скопированы:

```
nums := []int{1, 2, 3, 4, 5}

// создали слайс с длиной 0
numsCp := make([]int, 0)

// при копировании в пустой слайс ничего не произойдет
copy(numsCp, nums)

fmt.Println(numsCp) // []
```

Сортировка слайсов

Сортировка массива - распространненая задача в программировании. Во всех языках существуют готовые решения для этой задачи, и Go - не исключение. Стандартный пакет sort предоставляет функции для сортировки:

```
nums := []int{2,1,6,5,3,4}

sort.Slice(nums, func(i, j int) bool {
    return nums[i] < nums[j]
})

fmt.Println(nums) // [1 2 3 4 5 6]
```

Сортировка слайсов

Рассмотрим функцию Slice(x interface{}, less func(i, j int) bool). В описании функции присутствует неизвестный тип данных interface{}. Следует запомнить, что пустой интерфейс interface{} в Go означает тип данных, под который подходит любой другой тип.

```
func Print(arg interface{}) {
    fmt.Println(arg)
}

func main() {
    Print("hello!")
    Print(123)
    Print([]int{1,5,10})
}
```

```
hello!
123
[1 5 10]
```

Сортировка слайсов

То есть в функцию Slice(x interface{}, less func(i, j int) bool) передается слайс любого типа данных, как первый аргумент. Вторым аргументом передается функция, которая берет элементы по индексу и определяет должен ли элемент по индексу i находиться перед элементом по индексу j.

Сортировка слайсов

"Под капотом" в функции sort.Slice используется быстрая сортировка. В пакете также присутствует сортировка вставками sort.SliceStable:

```
nums := []int{2,1,6,5,3,4}

sort.SliceStable(nums, func(i, j int) bool {
    return nums[i] < nums[j]
})

fmt.Println(nums) // [1 2 3 4 5 6]
```

Сортировка слайсов

Выбор алгоритма зависит от набора и размера данных, архитектуры процессора, скорости доступа к памяти, то есть от многих факторов. Для большинства стандартных случаев используется sort.Slice, пока производительность или нестабильность алгоритма не станет "узким горлышком".

Map

Map — тип данных, предназначенный для хранения пар ключ-значение. В других языках эту структуру так же называют: хэш-таблица, словарь, ассоциативный массив. Запись и чтение элементов происходят в основном за O(1)

```
// создание пустой мапы
var m map[int]string

// сокращенное создание пустой мапы
m := map[int]string{}

// рекомендуемое создание с обозначением
размера
m := make(map[int]string, 10)

// создание мапы с элементами
m := map[int]string{1: "hello", 2: "world"}

// добавление элемента
m[3] = "!" // map[1:hello, 2:world, 3:!]

// чтение элемента
word := m[1] // "hello"
```

Map

При чтении элемента по несуществующему ключу возвращается нулевое значение данного типа. Это приводит к ошибкам логики, когда используется bool как значение. Для решения данной проблемы при чтении используется вторая переменная, в которую записывается наличие элемента в мапе:

```
existedIDs := map[int64]bool{1: true, 2: true}

idExists, elementExists := existedIDs[2] // true, true

idExists, elementExists := existedIDs[225] // false, false
```

Map

Элементы удаляются с помощью встроенной функции delete(m map[Type]Type1, key Type):

```
engToRus := map[string]string{"hello":"привет", "world":"мир"}

delete(engToRus, "world")

fmt.Println(engToRus) // map[hello:привет]
```

Map

Мапы в Go всегда передаются по ссылке:

```go
package main

import (
    "fmt"
)

func main() {
    m := map[int]string{1: "hello", 2: "world"}

    modifyMap(m)

    fmt.Println(m) // вывод: map[1:changed 2:world 200:added]
}

func modifyMap(m map[int]string) {
    m[200] = "added"

    m[1] = "changed"
}
```

Обход мап

Как и слайс, мапу можно обойти с помощью конструкции for range:

```
idToName := map[int64]string{1: "Alex", 2: "Dan", 3: "George"}

// первый аргумент — ключ, второй — значение
for id, name := range idToName {
    fmt.Println("id: ", id, "name: ", name)
}
```

Обход мап

Стоит учитывать, что порядок ключей в мапе рандомизирован:

```go
numExistence := make(map[int]bool, 0)

// записали по порядку числа от 0 до 9
for i := 0; i < 10; i++ {
    numExistence[i] = true
}

// обходим мапу и выводим ключи
for num := range numExistence {
    fmt.Println(num)
}
```

```
8
1
2
3
6
7
9
0
4
5
```

Руны

В современном мире невозможно работать только со строками, состоящими исключительно из ASCII символов. Везде используются нестандартные знаки, языки отличные от латиницы и эмодзи. Для работы с такими Юникод символами в Go представлен тип данных rune:

```
package main

import (
    "fmt"
)

func main() {
    emoji := []rune("привет😀")

    for i := 0; i < len(emoji); i++ {
        fmt.Println(emoji[i], string(emoji[i])) // выводим код символа и его строковое представление
    }
}
```

Руны

rune — это алиас к int32. Как и байты, руны были созданы для отличия от встроенного типа данных. Каждая руна представляет собой код символа стандарта Юникод. Строка свободно преобразуется в []byte и []rune, но эти 2 типа данных не конвертируются между собой напрямую:

```
s := "hey😉"

rs := []rune([]byte(s)) // cannot convert ([]byte)(s) (type []byte) to type []rune

bs := []byte([]rune(s)]) // cannot convert ([]rune)(s) (type []rune) to type []byte
```

Руны

В Go присутствует синтаксический сахар при обходе строки. Если использовать конструкцию for range, строка автоматически будет преобразована в []rune, то есть обход будет по Юникод символам:

```
package main

import (
    "fmt"
)

func main() {
    emoji := []rune("cool😀")

    for _, ch := range emoji {
        fmt.Println(ch, string(ch)) // выводим код символа и его строковое представление
    }
}
```

Оператор ...

- Оператор ... указывается перед типом данных и упаковывает все параметры указанного типа в слайс элементов этого же типа - тип возвращаемого значения об этом явно говорит.
- Распаковать слайс, чтобы элементы можно было передать в качестве параметров в такую же вариативную функцию, можно тоже при помощи оператора ..., который указывается после переменной.

Вариативные функции

Последний аргумент функции может быть вариативным. Функция может иметь максимум один вариативный аргумент и этот аргумент всегда слайс. Чтобы обозначить аргумент вариативным, нужно поставить три точки ... перед его типом:

```go
package main

import (
    "fmt"
)

func main() {
    // кол-во аргументов может быть любым
    PrintNums(1, 2, 3)
}

func PrintNums(nums ...int) {
    for _, n := range nums {
        fmt.Println(n)
    }
}
```

Вариативные функции

Также тремя точками можно разбить слайс на элементы при передаче в вариативную функцию. Например, встроенный метод append(slice []Type, elems ...Type) []Type, который добавляет последний элемент в слайс, принимает вариативный аргумент elems ...Type. Чтобы добавить один слайс в конец другого, нужно разбить второй слайс на элементы путем добавления трех точек ... после переменной:

```
nums1 := []int{1,2,3,4,5}

nums2 := []int{6,7,8,9,10}

res := append(nums1, nums2...) //[1 2 3 4 5 6 7 8 9 10]
```

Будут вопросы? Не стесняйтесь!